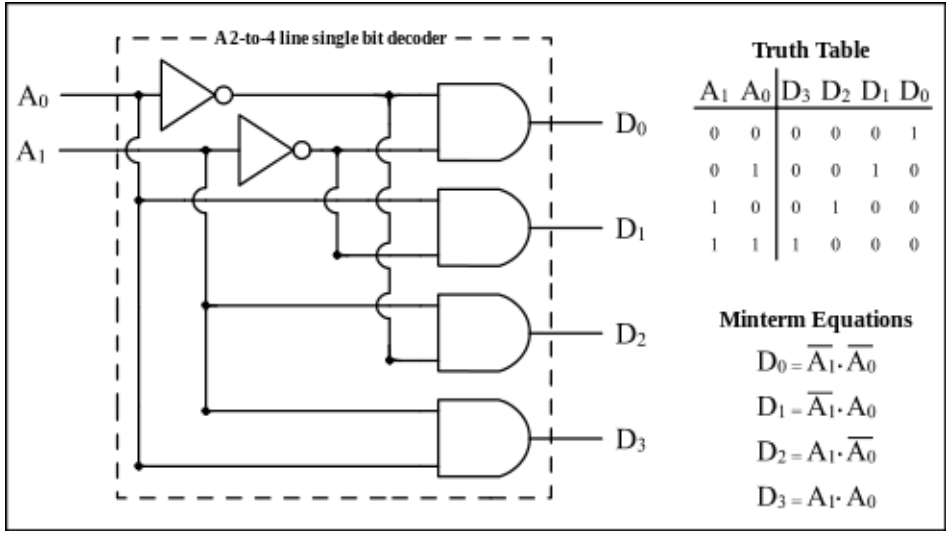
GPU Architecture meeting 2

BE BOUNDLESS

Additional Hardware

As promised last time

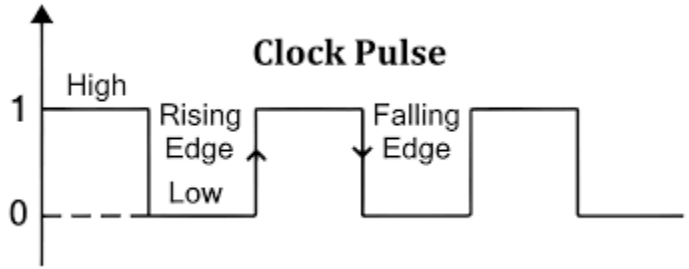
> **Decoder**
  - **Changes N width input into 2^N width, with single bit enabled**
  - **Useful for selecting one of 2^N from N bits**

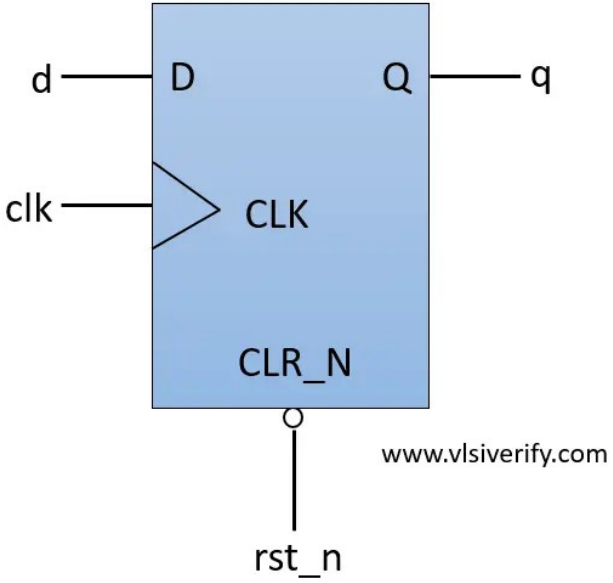


Additional Hardware

Hugely important!!!!

> **D- FlipFlop**
  - **Holds onto input for whole clock cycle**
  - **Takes new input @ (positive or negative) clock edge**
  - **Enables a lot of computing**





Asynchronous active low reset D flip flop

Today's Goals

Learn all about the next pieces of our processor

- **Instructions and Instruction Cycle**
  - **What/why**
  - **Instruction set**
- **Registers**
- **ALU**

Exercise

What does this number mean?

`10001011000000100000000001000001`

Binary Refresher

0 and 1 only, pay attention to places

1101

> **To understand what number this is, we can take a sum of 2^x, where x is the place of the number**

Binary Refresher

0 and 1 only, pay attention to places

Place	3	2	1	0
Value	1	1	0	1

> We have a 1 in the zero's, two's, and three's places
  - So we can sum this as $(1)2^0 + (0)2^1 + (1)2^2 + (1)2^3$
    - $1 + 0 + 4 + 8 = 13$

Binary Refresher

What about negative numbers?

- Sign Magnitude
  - Most Significant Bit denotes sign
  - Rest denotes magnitude
- Two's Complement
  - Read the same, but different process
  - Flip all bits, add 1
  - This is how most computers do it because it makes subtraction (more on this later)

Binary Refresher

So 1101 can be

- Read as is: 1101 = 13
- Sign Magnitude: 1101 = - (2^2+2^0) = -5
- Two's Complement: 1101 = - (0010 + 1) = - (0011) = -3
- For our purposes, use online converters if necessary

Exercise (cont.)

Maybe we know what this is now?

> 10001011000000100000000001000001
>
> $2^{31}+2^{27}+2^{25}+2^{24}+2^{17}+2^{6}+2^{0}=2332164161$
>
> **Maybe it's negative?**
>
> - **SM: -184680513**
> - **2's comp: -1962803135**
>
> **Or maybe this isn't just a really big number**

Assembly Languages

Processor's toolbox

- **Near 1:1 with machine code**
  - **Still kind of human readable**
- **Designed and licensed out for processor dev**
  - **Intel's x86, ARMv8**
  - **AMD RDNA and NVIDIA PTX for GPUs**
- **Instruction Set Architecture**
  - **Every Instruction the processor can do**

```
.MODEL SMALL
.STACK 100H
.CODE

MOV AX, 0x3C
MOV BX, 0000000000001010B
ADD AX, BX
MOV BX, 14
SUB AX, BX

MOV AH, 04CH
INT 21H
```

Assembly Languages cont.

Basics

- **Opcode – binary designated as a instruction type, bit width varies**
  - **ADD, SUB, for example**
- **Rd, Rm, Rn – Register operands**
  - **Rd is the destination**
  - **Rn and Rm are sources (nth, mth)**
- **When written, registers denoted as X(number of register)**

Exercise (cont.)

So maybe this is assembly?

```
10001011000000100000000001000001
```

Exercise (cont.)

So maybe this is assembly? (ARM)

```
>[10001011000][00010][000000][00010][00001]
>[      Opcode][    Rm][  Shamt][    Rn][    Rd]
>[         ADD][      ][       ][      ][      ]
```

Exercise (cont.)

So maybe this is assembly? (ARM)

```
>[10001011000][00010][000000][00010][00001]
>[     Opcode][   Rm][ Shamt][   Rn][   Rd]
>[        ADD][   X2][     0][   X2][   X1]
        > This is written in ARM as:
              > ADD X1, X2, X2
```

Expanding on the Instruction Cycle

Parts of a task that a computer needs to complete

- **Instruction Fetch**
- **Register / Decode**
- **Execute**
- **Memory access**
- **Write back**

Instruction Fetch

All instructions are kept in what's called instruction memory

- **Essentially a queue of instructions to be completed**
- **Pull a new one every clock cycle**
- **Length of instruction depends on design**

Registers/Decode

Given an instruction from ifetch

> **Continuously hold data across clock cycles**
> **From 2^N registers, output the data from one**
> **Be able to single out a register out of 2^N to write to from N bits**

Execute

Given two data inputs of [ ] length + the instruction

> **Correctly compute addition, subtraction, and logical operations**

> **Raise flags given correct circumstance**

> **Select output based on encoded instruction**

Team split

All together	
• Decide ISA (instruction dimension impacts every other decision)	
Register	ALU
• Make Registers that hold data • Make Mux that selects and outputs data from register • Make decoder that selects a register to write	• Make logic necessary for arithmetic (adder), and any desired logic • Make mux that chooses output